

PROJECT TESTING PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Testing Overview

The Project Testing Phase focuses on verifying that the ComicCraft application works correctly as an end-to-end comic generation system. Testing covers the AI text generation workflow, image creation, layout construction, PDF export, FastAPI routes, frontend templates, and local deployment.

2. Testing Objectives

The testing process is intended to verify that:

- User inputs are correctly received by the application.
- The comic outline is generated in the expected structured form.
- Narration and character dialogues are generated from the outline.
- Images are generated from the corresponding image prompts.
- Images and story text are correctly matched into comic panels.
- The comic preview is rendered through the web interface.
- The final comic can be exported as a PDF.
- The FastAPI application and API endpoints operate correctly during local testing.

3. Testing Scope

The project document specifically describes local deployment testing to verify the complete workflow, including AI text generation, image creation, layout building, and PDF export before future cloud deployment is considered.

4. Testing Approach

Testing is organized around the individual application components and then the complete generation workflow. The project also provides a dedicated /test-image route and FastAPI's /docs interface to support API and image-generation verification.

PROJECT TESTING PHASE – INPUT AND AI GENERATION TESTING

1. User Input Testing

The homepage collects five primary inputs: story prompt, main character name, setting, story tone, and art style. Testing verifies that these values are captured correctly by the FastAPI backend.

Input fields to verify:

- Story Prompt
- Main Character Name
- Setting
- Story Tone
- Art Style

2. Structured Outline Testing

The `generate_outline()` function in `gemini_flash.py` is tested as the first AI stage. The expected result is a structured five-panel outline where each panel contains a panel number, title, scene description, and image-generation prompt.

Expected verification points:

- Five-panel structure is produced.
- Each panel has the required descriptive information.
- The outline reflects the user's story prompt and preferences.

3. Story and Dialogue Testing

The `generate_story()` function in `gemini_pro.py` is tested after outline generation. The function should expand the outline into a formatted comic story containing narration and character dialogues.

Verification points:

- Story content corresponds to the generated outline.
- Narration is present for the generated comic.
- Character dialogue is included where appropriate.

PROJECT TESTING PHASE – IMAGE, LAYOUT AND EXPORT TESTING

1. Image Generation Testing

The `generate_image()` function in `image_generator.py` is tested separately and as part of the full comic workflow. It receives an image prompt and uses Stable Diffusion to generate a comic-style illustration.

Image testing checks:

- A valid image is generated from the supplied prompt.
- The generated image is saved to the `static/panels` directory.
- The returned file path can be used by later application stages.

2. Dedicated Image Test Route

The `/test-image` route provides a direct way to test image generation from a custom prompt without running the complete comic-generation pipeline. This separates image-generation verification from other application functions.

3. Comic Layout Testing

The `build_comic_layout()` function in `layout_builder.py` is tested to confirm that generated images are matched with their corresponding panel stories. The output should contain panel number, image path, and text.

4. PDF Export Testing

The `save_pdf()` function in `exporters.py` is tested to verify that the complete comic can be compiled into a multi-page PDF using FPDF. The generated document should contain the panel images and associated narrative content.

Expected export result:

- PDF file is created successfully.
- Panel content is placed in the exported document.
- The resulting file is saved in the configured `static/exports` folder.

PROJECT TESTING PHASE – API AND FRONTEND TESTING

1. FastAPI Route Testing

The main routes are tested through browser requests and the FastAPI interactive API documentation. The purpose is to verify that each route connects correctly to its intended backend function.

Route	Testing Purpose
/	Verify that the homepage and input form load correctly.
/generate	Verify complete form-based comic generation and preview.
/generate-comic/json	Verify JSON-based comic generation and response data.
/export-success	Verify the export-success page is displayed after download.
/test-image	Verify direct image generation from a custom prompt.

2. JSON API Testing

The `/generate-comic/json` endpoint accepts a JSON payload validated through the `PromptRequest` Pydantic model. Testing verifies that valid JSON input is accepted and that the endpoint returns comic layout data and PDF information.

3. Frontend Template Testing

Jinja2 templates are tested to confirm that generated backend data is correctly rendered. The `comic_preview.html` page should display panel number, title, image, scene description, captions, and narration.

4. Error Handling Testing

The project uses `HTTPException` and `try-except` blocks in route handlers. Testing includes verifying that unexpected API failures and JSON decoding errors are handled without breaking the application flow.

PROJECT TESTING PHASE – LOCAL DEPLOYMENT AND FINAL VERIFICATION

1. Local Deployment Testing

The final testing stage runs ComicCraft locally using the FastAPI development server. The project document specifies Uvicorn with the application entry point and hot reload enabled.

uvicorn app.main:app --reload

After starting the server, the application is accessed through <http://127.0.0.1:8000>. The /docs interface is used to explore and test available API endpoints.

2. End-to-End Workflow Verification

The complete workflow is checked in the following sequence:

1. Open the ComicCraft homepage.
2. Enter the story prompt and comic preferences.
3. Submit the form to the /generate route.
4. Verify the structured comic outline and generated story.
5. Verify that panel illustrations are generated.
6. Verify that images and story content are organized into the comic layout.
7. Verify the comic preview page.
8. Download and verify the generated PDF.
9. Verify the export-success page.

3. Final Testing Checklist

- ✓ User input processing verified.
- ✓ Gemini outline-generation workflow checked.
- ✓ Gemini story and dialogue workflow checked.
- ✓ Stable Diffusion image-generation workflow checked.
- ✓ Layout construction checked.
- ✓ PDF export checked.
- ✓ FastAPI routes and Jinja2 rendering checked.
- ✓ Local deployment workflow verified.

Conclusion

The Project Testing Phase verifies the major components and the complete ComicCraft workflow before future deployment. It provides a structured process for checking input handling, AI generation, image creation, layout construction, web rendering, API routes, and PDF export.